

May 2026 – Aug 2026

Anduril Industries AI Grand Prix

State Estimation GNC Computer Vision Sensor Fusion System Identification Python



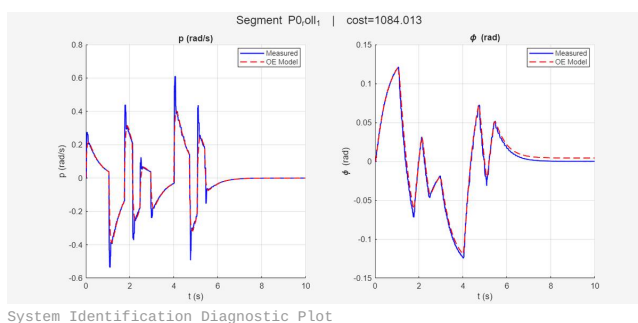
What — A fully autonomous drone pilot stack built for the Anduril Industries AI Grand Prix competition, designed to navigate a racing course from takeoff to landing without any human input.

How — Developed MAVLink communication handlers for telemetry and command, a computer vision pipeline for gate detection, state estimation (Kalman Filter on IMU+CV data), and a cascade controller architecture managing thrust and attitude. Conducted a formal system identification study of the drone's flight dynamics to help guide control design.

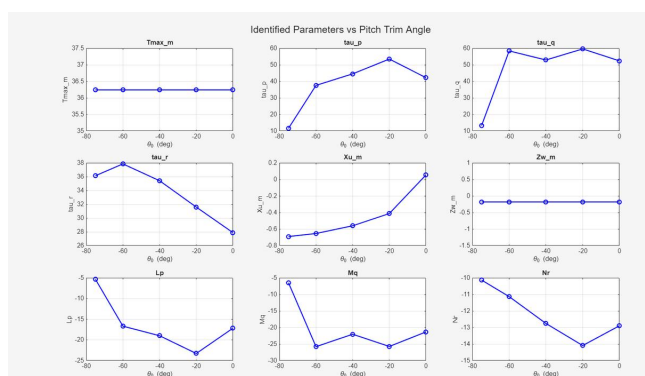
Results — Advanced to the final round of virtual qualifiers, but did not make it to physical competition. The system achieves stable autonomous flight with real-time gate detection and course navigation.

Challenges — The simulator registered drone thrust as a series of large, isolated impulses rather than a smooth continuous force, making raw IMU-based velocity estimation extremely noisy and unreliable. A buffer-based smoothing approach was implemented to manage the impulses, and a system identification study of the thrust characteristics was attempted to better model the dynamics. Ultimately, combining IMU estimates with computer vision measurements was necessary to get reliable velocity and state estimates.

A separate and more disruptive issue involved the simulator's interpretation of MAVLink commands. Bitmask 7 is documented to set a desired attitude target while ignoring angular rate inputs, but the simulator consistently treated it as an angular rate command instead. A full controls architecture was built around that behavior, only for the simulator to silently switch its interpretation to match the official documentation partway through the competition. This effectively invalidated a large portion of the controls work already done. The controls were rebuilt to align with the corrected behavior.



System Identification Diagnostic Plot



Multi-Regime Parameter Visualization

[GitHub](https://github.com/ayushc121/AndurilGP) <https://github.com/ayushc121/AndurilGP>[Video](https://www.youtube.com/watch?v=HfdJw55CzkQ) <https://www.youtube.com/watch?v=HfdJw55CzkQ>

Aug 2026

Embedded GPS-Aided Navigation System

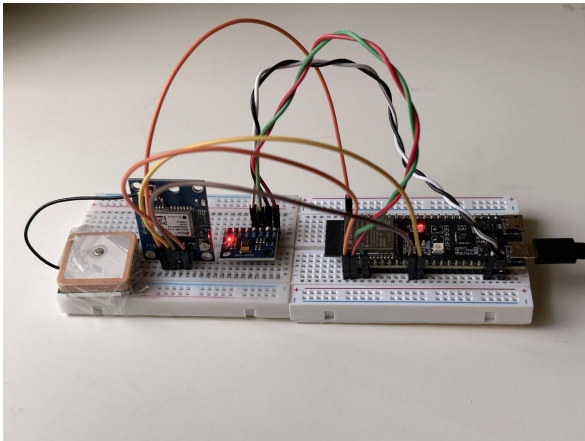
State Estimation

Kalman Filter

Embedded Systems

C++

Signal Processing



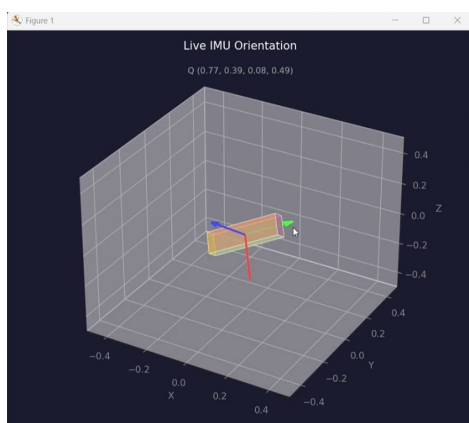
What — A from-scratch inertial navigation system on an ESP32-S3 microcontroller, fusing high-rate IMU data with low-rate GPS to produce real-time estimates of position, velocity, and orientation. All filter algorithms are implemented by hand in embedded C++, without any external matrix libraries.

How — A two-stage architecture keeps the system fully linear. The first stage runs a Madgwick AHRS filter to track orientation as a quaternion, fusing gyroscope integration with accelerometer-based gravity correction and GPS course bearing for yaw at speed. The second stage runs a 9-state linear Kalman filter (position, velocity, and accelerometer bias per axis) that uses the Madgwick quaternion to rotate body-frame accelerations into world-frame ENU coordinates before integrating them. GPS provides absolute position corrections at 4 Hz, and a Zero Velocity Update (ZUPT) is triggered during stationary periods to prevent velocity error from accumulating freely. Sensor calibration runs at startup: 500 IMU samples average out gyro bias, and 50 GPS fixes establish the ENU origin. A Python telemetry script renders a live 3D visualization of both orientation and position simultaneously.

Results — When stationary, the Kalman position estimate converges to within roughly 1 meter of the starting point. Pitch and roll from the Madgwick filter are solid with no observable drift. Yaw drifts slowly without a magnetometer, corrected only during GPS movement above 2 m/s. The system is genuinely useful for smooth trajectory logging, short-gap dead reckoning across brief GPS outages, and attitude monitoring. Sub-10-meter dynamic tracking is a hardware limitation of the consumer GPS noise floor, not a filter design limitation.

Challenges — Electromagnetic interference was a practical concern from the start. The MPU6050 runs I2C at high speed and is sensitive to common-mode noise picked up by the wiring. Braiding the wires to the IMU provided a meaningful reduction in accelerometer noise and was a simple but important hardware fix.

A more subtle bug involved the initialization of the IMU timestamp. The variable tracking the previous IMU read time was initialized to zero, but the main loop does not actually start reading sensor data until after startup calibration completes: 500 gyro samples plus 50 GPS fixes, which takes well over 10 seconds. On the very first IMU read, the computed time delta was therefore the entire boot duration rather than a normal sample interval. This massive dt fed directly into the gyro integration step of the Madgwick filter, rotating the orientation quaternion by an enormous amount and producing a wildly incorrect initial orientation. Since the Kalman filter uses that quaternion to rotate accelerometer data into the world frame and subtract gravity, an incorrect orientation at initialization caused gravity to be decomposed incorrectly, injecting a spurious acceleration component into the position estimate and driving significant drift in the first few filter iterations before the Madgwick filter could correct itself. The fix was straightforward once the cause was identified: setting the initial timestamp at the moment the first IMU sample is actually read rather than at program start.



Orientation Visualization

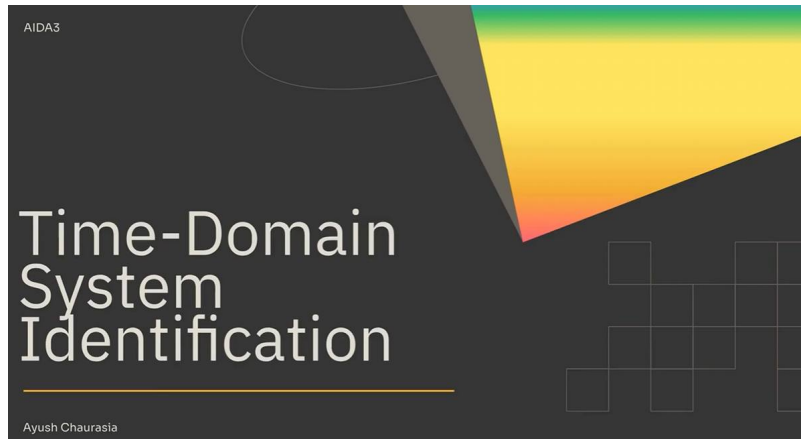
GitHub ➤ <https://github.com/ayushc121/kalmanFilter>

Video ➤ <https://www.youtube.com/watch?v=lAF7UrxTVgM>

Jan 2026 – May 2026

3DoF Plane System Identification Study

Python MATLAB System Identification Modeling & Simulation Research

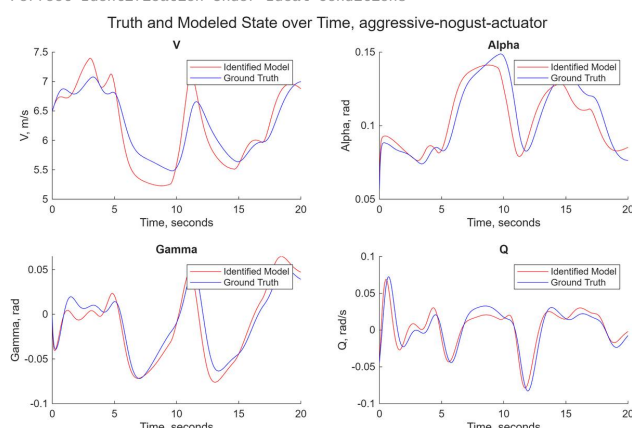
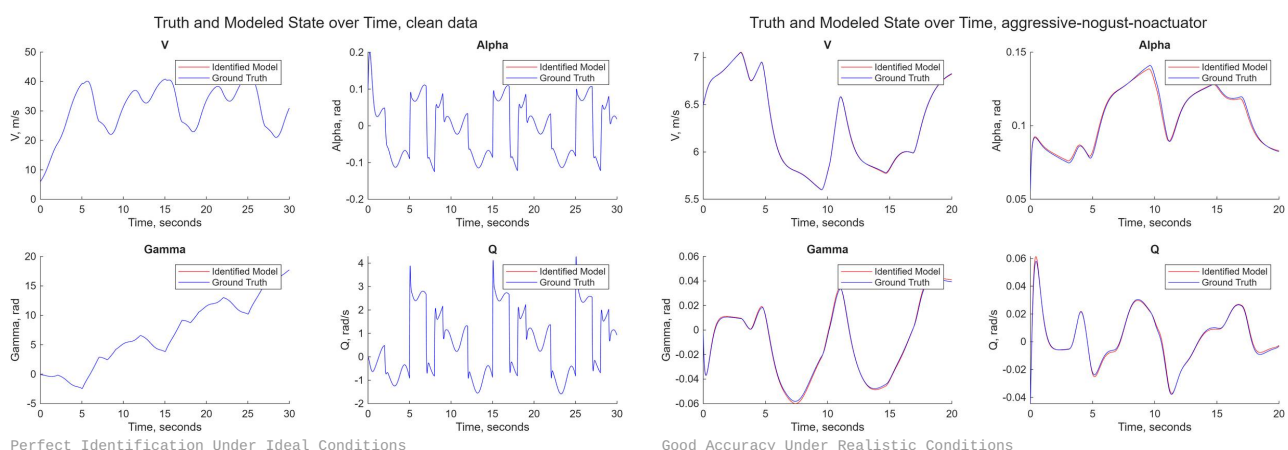


What — A research study evaluating system identification methods for 3-DoF aerodynamic models under noisy and data-limited conditions representative of real flight test environments.

How — Reviewed existing literature and implemented NASA's SIDPAC system identification framework. Built a custom Python-based 3-DoF flight dynamics engine to generate controlled, high-fidelity datasets, then used it to simulate eight targeted edge-case scenarios under external disturbances, enabling systematic evaluation of estimator performance across degraded data conditions.

Results — Achieved sub-3% parameter estimation error in noisy 3-DoF aerodynamic simulations with external disturbances using the SIDPAC approach. The custom simulation engine improved testing reliability and provided a repeatable benchmark environment for comparing identification methods.

Challenges — The core 3-DoF equation structure was relatively well-defined given the standardized aerodynamic formulations available in the literature, so implementing the baseline was tractable. The more difficult part was augmenting the simulation with realistic degradations: adding atmospheric gust disturbances and actuator delay introduced enough additional dynamics that the SIDPAC estimator began struggling to converge consistently on the disturbed datasets. Several variations were attempted to model these effects more faithfully, including explicit actuator delay modeling and gust shaping, but reliable convergence under the more aggressive combined conditions proved elusive. This points to a genuine limitation of the approach: more sophisticated algorithms, such as multi-shot or output-error methods with better initialization, may be better suited to these conditions and would be a natural next direction for the study.

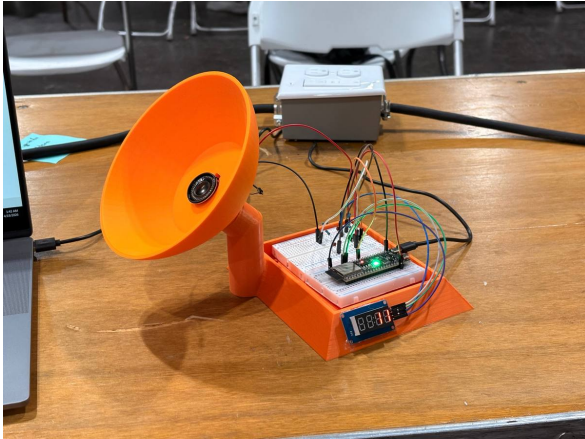


GitHub ➤ <https://github.com/ayushc121/3dofSysID>

Video ➤ https://www.youtube.com/watch?v=ov8d1u_JZXE

Apr 2026

Acoustic Drone Detection System

Embedded Systems C++ Signal Processing Machine Learning PyTorch Prototyping

What — A low-cost, fully standalone acoustic early-warning sensor for nearby drone detection, designed to operate entirely offline with no cloud or internet dependency.

How — Repurposed an 8Ω speaker as a directional microphone through a custom breadboarded circuit. Programmed an ESP32 microcontroller in C++ to perform real-time FFT analysis on the incoming audio stream, driving hardware alerts with zero latency. Trained a PyTorch neural network on drone acoustic signatures, then embedded the quantized model weights directly onto the microcontroller for on-device inference.

Results — Achieved 85% drone detection accuracy at 15 yards. The complete system (from circuit design through trained model deployment) was built in 36 hours.

Challenges — No dedicated microphone was available during the build, and the off-the-shelf sound detector modules on hand performed poorly at the frequency range characteristic of drone motor noise. The workaround was repurposing a speaker as a directional microphone by building a custom amplifier and filtering circuit around it, which gave usable sensitivity in the right frequency band. Getting a clean signal was another significant problem: the improvised microphone picked up ambient noise aggressively and the raw input was very noisy. Rather than trying to solve this entirely in the signal processing layer, the noise robustness was largely offloaded to the neural network itself. Training on noisy samples meant the model learned to ignore the irrelevant frequency content and focus on the drone signature, which turned out to be an effective approach given the hardware constraints and the 36-hour timeline.



Successful Detection of Drone Audio

GitHub ➤ <https://github.com/ayushc121/starkHacks>

Video ➤ <https://www.youtube.com/watch?v=E8IrfMaYsBM>